Job Scout.

Job Scout: how it is built

OCTOBER 2026 · VERSION 0.14 · BUILT WITH CLAUDE

What it is

Job Scout is an automated job search. It runs every morning and does six things:

1. It reads my job alerts and the replies to my applications.
2. It finds and reads the full job adverts.
3. It scores each role against criteria that I wrote.
4. For the best few roles, it drafts a CV and a covering letter.
5. A second AI agent checks every claim in those drafts against my career record.
6. It puts the results on a private board, where I decide what to do.

It never applies for a job, replies to an email or contacts anyone. I do those things myself. It has run every day since 14 October 2026.

I designed the system and made every decision. Claude, Anthropic's AI model, wrote the code and the agents' instructions to my specification. The work was done in eight sprints (short, planned blocks of work). Each sprint had agreed test cases before building started, and was checked against them at the end.

Using AI to write a covering letter is not new. The interesting part is the design: how to make a system of AI agents safe enough to run every day on my own mailbox and career record. Three design choices make this possible. Words in *italics* are explained at the end.

Design choice 1: use an AI agent only where it is needed

Each extra *AI agent* adds cost, more hand-over points and more ways to fail. So a step becomes a separate agent only for one of three reasons:

- **Isolation.** The step reads text written by strangers. So it must hold nothing valuable, and it must have no tools that could cause harm.
- **Independence.** The step judges the work of another step. So it must be separate from the step that did the work.
- **On demand.** The step must be able to run on its own.

All other steps are normal code, not AI. This includes reading the mail, filtering, removing duplicates, counting *ATS* keywords and every check on the AI's output. The result is three AI agents around a core of code: a local reader, a writer and an independent reviewer.

Design choice 2: each kind of data has a boundary

The design starts from one question: which part of the system may see which data?

DATA	WHO CAN SEE IT
My mailbox	Only code on the local PC. The code can read mail, but it cannot change or send anything.
Job adverts and replies (written by strangers)	A *local model* on the local PC, with no tools and no internet access. Only a checked summary leaves the PC.
My career record	The writer and the reviewer, in the cloud. The reviewer can read but not write.
My mail password	Nothing. It is encrypted for my Windows account and kept outside every folder an AI can reach.

I chose a local model for safety, not to save money. An advert may contain hidden instructions (*prompt injection*). But the model that reads the advert has no tools, so it cannot act on them. After that, code keeps only the facts that appear in the advert's own words.

Design choice 3: several layers of protection

No single control is trusted alone. There are five layers:

1. **Text is treated as data.** Before any AI sees an alert, advert or reply, code removes lines that look like instructions and reports what it removed. The AI is told to treat the remaining text as material to assess, never as instructions.
2. **Code checks before anything is saved.** Code checks every score, draft, claim, reply match and board entry before it is written. If a check fails, nothing is written and the problem is reported.
3. **The reviewer must show evidence.** For each claim it accepts, the reviewer must quote my career record. Code then confirms that the quote really exists. So the review never depends on the reviewer's word alone.
4. **Changes to my board must be earned.** At first, the system can only suggest changes. It may move a card on my board by itself only after I have tested its reply reader on my own replies. Every move is logged and can be undone.
5. **Limits and an off switch.** There is a pause switch and a daily limit on adverts read and drafts written. A log records every step: which model ran, with which inputs and instructions.

Testing: results are measured, not assumed

The figures below come from the test reports, 17 to 21 October 2026.

- **Local reader.** In a test of 22 adverts, it extracted the requirements and pay correctly for 21, the same in each of three runs (21 October). A first automatic check, on 19 October, gave 18 of 22. The investigation found that the test was at fault, not the model: it was too strict about word endings, one advert had in fact been removed (the reader rightly marked it closed), and one expected answer depended on a heading that the model no longer copied. The one real miss: on a long advert, the model stops before the last two requirements. Each morning, code records the model's version fingerprint and the runtime version. If the model changes without notice, for example after an automatic update, the system gives a warning before it uses it.
- **Reviewer.** It passed all eight test cases. Each case contained one planted problem: for example, a planned result written as an achievement, a claim I have decided never to make, or an instruction hidden in a keyword.
- **Red team.** *Red teaming* is a practice from security testing: someone plays the attacker to find weaknesses before a real attacker does. Each quarter, ten hostile, made-up inputs go through the real system (first run 17 October 2026). They include hidden instructions in alerts, email subject lines and adverts, a request to send data outside, a fake recruiter, and a fake job offer that asks for bank details. All ten attacks failed. The exercise also found four bugs, all now fixed:
- an instruction in an email subject line reached the cloud without being removed;
- the cloud part could not reach the test results;
- a warning on a reply ("asks for bank details") did not appear on the board;
- in a calendar invitation, the time-zone information was read as the meeting time.
- **Scoring test.** This *evaluation* compares the system's scores for 34 roles with my own decision on each: apply or not. The first run taught me more about the test than about the scoring. For most roles, the full advert had not been saved, so the system scored them again from the short alert only. The original scores agreed with my decisions for 24 of 34 roles (71%). The new scores agreed for only 11 of 34. I am now rebuilding the test from runs that save every advert. Only then will I set a *baseline*.
- **Automated tests.** There are 151 tests for the code and 164 for the checking tools. They use made-up emails and adverts, and several of them are hostile.

A one-page risk register lists each risk, its control, its owner and the date it was last checked. It follows the four functions of the NIST AI Risk Management Framework (from NIST, the US standards agency): Govern, Map, Measure and Manage. I review it at the end of every sprint.

Lessons from daily use

Two useful lessons came from running the system every day, not from the design.

First, the local PC's power-saving mode (Windows "Modern Standby") blocked the scheduled wake-up and turned off the network, so the morning runs started late. The fix was simple: the local PC no longer sleeps when it is plugged in.

Second, a graphics setting made the local model run on the main processor instead of the graphics card, at about one tenth of its normal speed. Only the reader's own speed warning showed this.

Both fixes are now in the setup notes, and the reader now stops by itself when the model is slow, leaving the rest to the cloud. My conclusion: AI systems often fail where they meet the real world, not only in their reasoning. They need the same operational care as any other service.

How it was built

- **Process.** The work was done in sprints. Before each sprint, the work was written as a specification, with numbered test cases and my answers to open questions. The tests were written before the code. At the end, each sprint was checked against a live run. Unfinished work went back to the *backlog* and was ranked again.
- **Independent code review.** Since Sprint 7, before each delivery, a separate AI agent that has not seen the work reviews the code. In Sprint 7 it found eleven issues. Nine were fixed before delivery, and two were accepted as intended.
- **Technology.** PC part: TypeScript on Node.js, with no external libraries. Checking tools: Python, using only its standard library. Local reader: the qwen2.5:14b model, run with Ollama on an NVIDIA RTX 5080 graphics card. Cloud part: scheduled tasks in Claude. Board: a private web page built in Claude, with its own database.

The architecture diagrams, the decision log and the risk register are kept with the code.

Terms used in this note

TERM	MEANING
AI agent	An AI model with a task, instructions and a limited set of tools, which works on the task by itself.
ATS	Applicant tracking system: the software employers use to receive and filter applications, often by keywords.
Backlog	The ranked list of planned work.
Baseline	A reference result. Later results are compared with it to see if quality has changed.
Evaluation	A repeatable test that compares the AI's answers with answers a person has checked.
Local model	An AI model that runs on my own computer, not on the internet.
Prompt injection	Hidden instructions placed in text, for example in a job advert, to make an AI do something its owner did not intend.
Red teaming	Testing a system by playing the attacker, to find weaknesses before a real attacker does.